



UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Faculty of Engineering, Built Environment and
Information Technology

School of Information Technology
Department of Computer Science
COS326 – Database Systems
Practical 2 2026

Release Date: 12 August 2026
Submission Date and Time: 23 August 2026 @ 23:59Hrs
Lecturer: Mr S.M Makura
Total: 50 Marks

A. Objectives

1. Design an Enhanced Entity-Relationship Diagram (EERD) modeling class inheritance and complex relationships in an ORDBMS environment.
2. Implement PostgreSQL ORDBMS features including User-Defined Types (UDTs), domains, sequences, custom types, and table inheritance.
3. Write procedural SQL (PL/pgSQL) routines for business logic, search validation, array operations, and custom constraints.
4. Construct database triggers to enforce referential integrity across inherited tables and maintain immutable audit logging.

B. Submission Procedure

Submit a single compressed archive (uXXXXXXXXX_Surname_Initials(XXXXXXXXX is your student number) *e.g* u12345678_Smith_JN) containing the following files:

1. **EERD.pdf**: Diagram showing the database design.
2. **CreateStatements.sql**: All DDL for types, domains, sequences, tables, functions, and triggers.
3. **InsertQueries.sql**: All sample data population scripts (INSERT statements).
4. **SelectQueries.sql**: Analytical report queries (SELECT statements).
5. **TestTriggerQueries.sql**: Testing operations for triggers and integrity checks (INSERT, UPDATE, DELETE statements).
6. The completed **AI declaration form** where you detail the AI prompts you used and the usage types.

NO LATE submissions will be accepted after the submission date and time has lapsed. Do not wait until the last minute to submit and then provide reasons related to technical difficulties.

Academic Integrity: You may use AI-assisted tools (e.g., ChatGPT, Copilot, Gemini etc) to help you understand PostgreSQL concepts and SQL syntax. However, you remain responsible for the design, implementation, and correctness of your solution. During marking, you may be asked to explain or modify any part of your submission. Failure to demonstrate understanding may result in marks being deducted.

Question 1: CyberForensics 5G Incident Response Platform

Scenario

You have been recruited as a Senior Database Engineer by Aegis Digital Forensics. Aegis maintains an advanced incident response platform for monitoring 5G network slices and edge computing nodes.

Due to strict compliance standards, Aegis requires an object-relational database engine (PostgreSQL 18) capable of handling complex network data types, tracking investigator assignments, validating captured forensic evidence arrays, and maintaining an unalterable audit log of deleted incident records.

Source: Makura S.M (2026)

Database Conceptual Specification

1. Custom Types & Domains

- **NodeIPType (Domain):** A domain representing an IP address string limited to valid standard IPv4 address formatting (or text constrained to 15 characters).
- **SeverityType (ENUM):** Values: 'LOW', 'MEDIUM', 'HIGH', 'CRITICAL'.
- **InvestigatorName (UDT Composite):** Structured ROW type containing (title VARCHAR(10), first_name VARCHAR(50), last_name VARCHAR(50)).

2. Class / Table Inheritance Hierarchy

- **Incident (Base Table):**
 - incident_id (PK, Integer, generated by sequence incident_seq)

- `incident_code` (VARCHAR(10), Unique)
- `target_ip` (NodeIPType)
- `severity` (SeverityType)
- `detected_at` (TIMESTAMP)
- **NetworkIncident (Inherits Incident):**
 - `slice_id` (VARCHAR(20))
 - `evidence_hashes` (Text Array TEXT[]) — Stores an array of SHA-256 cryptographic hashes associated with captured network PCAP files.
- **NodeBreachIncident (Inherits Incident):**
 - `node_id` (VARCHAR(20))
 - `assigned_investigator` (InvestigatorName)
 - `is_contained` (BOOLEAN)

3. Auxiliary Lookup & Audit Tables

- **EvidenceCatalog (Table):**
 - `hash_id` (PK, generated by sequence `evidence_seq`)
 - `evidence_hash` (TEXT, Unique) — Stores registered valid SHA-256 hashes.
 - `file_type` (VARCHAR(20))
- **DeletedIncidentAudit (Audit Table):**
 - Captures all attributes of deleted `NetworkIncident` and `NodeBreachIncident` rows, along with `deleted_at` (TIMESTAMP) and `deleted_by` (VARCHAR(50) user ID).

Task 1: Enhanced Entity-Relationship Diagram [5 Marks]

Destination File: EERD.pdf

Create a clean, well-structured Enhanced Entity-Relationship Diagram (EERD) using Draw.io or any similar platform that models the entire system domain.

- Model the base table Incident and its inherited child tables (NetworkIncident and NodeBreachIncident) showing the specialization/generalization relationship.
- Display composite user-defined types (UDTs), array attributes, and domains clearly.
- Include primary keys, sequences, and relationships to auxiliary/audit tables.
- Include your full student name and student number within the PDF document.

Task 2: DDL & Database Objects Creation [23 Marks]

Destination File: CreateStatements.sql

Write all PostgreSQL DDL scripts and PL/pgSQL code to build the database schema.

1. Domains, Enums, & Sequences (3 Marks):

- Domain NodeIPType, ENUM SeverityType, UDT InvestigatorName, and sequences (incident_seq, evidence_seq).

2. Table Inheritance & Auxiliary Tables (5 Marks):

- Base table Incident, child tables NetworkIncident and NodeBreachIncident using INHERITS.
- Lookup table EvidenceCatalog and audit table DeletedIncidentAudit.

3. PL/pgSQL Utility & Validation Functions (8 Marks):

- `format_investigator_name(InvestigatorName)` : Returns a formatted text string (e.g., 'Dr. Alex Mercer').
- `is_hash_registered(TEXT)` : Returns BOOLEAN indicating if an evidence hash exists in EvidenceCatalog.

- `has_duplicate_hashes(TEXT[])`: Returns BOOLEAN checking if the input array contains duplicate hash strings using nested loop structures (e.g., nested FOREACH).
- `validate_evidence_array(TEXT[])`: Uses `is_hash_registered` and `has_duplicate_hashes` to return BOOLEAN checking that all array entries exist in `EvidenceCatalog` and no duplicates exist.

4. Trigger Functions & Triggers (7 Marks):

- **`check_evidence_integrity()` Trigger Function & Trigger:**
Attached to `NetworkIncident` (BEFORE INSERT OR UPDATE). Validates that `evidence_hashes` contains only valid, non-duplicate hashes from `EvidenceCatalog`. Raises an exception if validation fails.
- **`audit_deleted_incident()` Trigger Function & Triggers:**
Attached to `NetworkIncident` and `NodeBreachIncident` (AFTER DELETE). Automatically copies deleted row details, current timestamp, and the acting user ID (`CURRENT_USER`) into `DeletedIncidentAudit`.

Task 3: Data Population [4 Marks]

Destination File: `InsertQueries.sql`

Write standard SQL INSERT statements to populate the database tables with working data:

- Add at least 4 valid entries into `EvidenceCatalog`.
- Add at least 3 valid records into `NetworkIncident`.
- Add at least 2 valid records into `NodeBreachIncident`.

Task 4: Analytical SQL Reports [8 Marks]

Destination File: `SelectQueries.sql`

Write single, optimized SQL SELECT statements for the following operational reports:

1. **Investigator Assignment Summary (2 Marks):** List incident_code, target IP, containment status, and formatted investigator name (using format_investigator_name) for all breach incidents.
2. **Evidence Hash Lookup Query (3 Marks):** Given a specific evidence hash string (e.g., 'hash_101'), list all NetworkIncident records where this hash appears inside the evidence_hashes array.
3. **High-Risk Network Slices Report (3 Marks):** Group by slice_id and report the total number of incidents flagged with 'CRITICAL' or 'HIGH' severity on each slice.

Task 5: Trigger & Constraint Testing Operations [10 Marks]

Destination File: TestTriggerQueries.sql

Write DML queries (INSERT, UPDATE, DELETE) to explicitly test and prove that your triggers enforce integrity and capture audit logs as intended:

Test #	Operation & Destination Table	Tested Condition	Expected Result	Marks
TC-1	INSERT INTO NetworkIncident	Include an unregistered evidence hash in the evidence_hashes array.	Trigger aborts transaction with an error.	2
TC-2	INSERT / UPDATE on NetworkIncident	Provide an array with duplicate evidence hashes.	Trigger aborts transaction with an error.	2
TC-3	DELETE FROM NetworkIncident	Delete an existing network incident record.	Record deleted and verified inside DeletedIncidentAudit.	3
TC-4	DELETE FROM NodeBreachIncident	Delete an existing node breach incident record.	Record deleted and verified inside DeletedIncidentAudit.	3

Total Marks: 50